

TRƯỜNG ĐẠI HỌC FPT
KHOA CÔNG NGHỆ THÔNG TIN
(FPT University)

BÁO CÁO CUỐI KỲ

**Môn học: AI trong sản xuất
DevOps, DataOps, MLOps**

Mã môn: DDM501

Đề tài:

Xây dựng hệ thống MLOps end-to-end

Phân loại chất lượng rượu vang với giám sát, phát hiện drift
và tự động tái huấn luyện mô hình

Sinh viên thực hiện	Mã sinh viên
Phạm Minh Hoàng	25MS13285
Nguyễn Công Trọng	25MS13297
Ngô Minh Khôi	25MSA13236

Năm học: 2025 – 2026

Học kỳ: 1 – 2026

Hà Nội, tháng 10 năm 2026

Lời cam đoan

Chúng tôi xin cam đoan rằng báo cáo cuối kỳ này là công trình nghiên cứu và thực nghiệm của nhóm, được thực hiện dưới sự hướng dẫn của giảng viên môn học. Các số liệu, kết quả được trình bày trong báo cáo là trung thực và chưa từng được công bố ở bất kỳ công trình nào khác. Mọi tham khảo từ tài liệu nước ngoài đều được trích dẫn đầy đủ.

Hà Nội, tháng 10 năm 2026

Nhóm sinh viên thực hiện

Phạm Minh Hoàng Nguyễn Công Trọng Ngô Minh Khôi

Tóm tắt

Báo cáo này trình bày quá trình thiết kế và triển khai một hệ thống MLOps hoàn chỉnh (end-to-end) cho bài toán phân loại chất lượng rượu vang: dự đoán một mẫu rượu có *đạt chất lượng* (điểm cảm quan ≥ 6) hay không từ 11 chỉ số hóa lý, sử dụng bộ dữ liệu *UCI Wine Quality* (6.497 mẫu rượu đỏ và rượu trắng). Hệ thống bao gồm toàn bộ vòng đời của một mô hình học máy: từ thu thập dữ liệu, tiền xử lý, huấn luyện, phục vụ (serving), giám sát (monitoring), phát hiện data drift đến tự động tái huấn luyện.

Kiến trúc hệ thống được xây dựng với 8 dịch vụ Docker: **FastAPI** cho REST API serving, **MLflow** cho theo dõi thí nghiệm và quản lý mô hình, **Apache Airflow** cho orchestration pipeline tự động, **Evidently AI** cho phát hiện data drift, **Prometheus** và **Grafana** cho thu thập và trực quan hóa metrics, cùng **PostgreSQL** làm cơ sở dữ liệu nền.

Kịch bản thực nghiệm: mô hình ban đầu được huấn luyện trên rượu đỏ, sau đó dữ liệu production chuyển sang rượu trắng. Evidently phát hiện drift trên 10/11 đặc trưng và tự kích hoạt Airflow DAG; mô hình huấn luyện lại trên cả hai loại rượu đạt độ chính xác 76,1% trên tập kiểm tra, so với 70,7% của mô hình cũ trên cùng tập đó, nên được tự động đưa lên Production. Hệ thống vượt qua 96 test cases với độ bao phủ code 96% ($\geq 80\%$ theo yêu cầu). Báo cáo cũng phân tích tính công bằng của mô hình giữa hai loại rượu và khả năng giải thích mô hình.

Từ khóa: MLOps, data drift, model monitoring, Docker, FastAPI, MLflow, Apache Airflow, Evidently AI, Prometheus, Grafana, Wine Classification.

Mục lục

Lời cam đoan	1
Tóm tắt	2
1 Giới thiệu	7
1.1 Bối cảnh và động lực	7
1.2 Mục tiêu đề tài	7
1.3 Phạm vi thực hiện	7
1.4 Cấu trúc báo cáo	8
2 Cơ sở lý thuyết	9
2.1 MLOps là gì?	9
2.1.1 Các thành phần cốt lõi của MLOps	9
2.2 Data Drift và Model Drift	9
2.3 Các công cụ sử dụng	10
2.3.1 MLflow	10
2.3.2 Apache Airflow	10
2.3.3 FastAPI	10
2.3.4 Evidently AI	10
2.3.5 Prometheus & Grafana	10
2.3.6 Docker & Docker Compose	10
3 Kiến trúc và Thiết kế Hệ thống	11
3.1 Tổng quan kiến trúc	11
3.2 Luồng dữ liệu tổng thể	11
3.3 Cấu trúc thư mục dự án	12
3.4 Thiết kế Airflow DAG	13
3.5 Kiến trúc CI/CD	13
4 Mô tả Dữ liệu, Mô hình và Responsible AI	14
4.1 Dataset: Wine Quality	14
4.1.1 Mô tả 11 features	14
4.2 Mô hình học máy	15
4.2.1 Lựa chọn mô hình	15
4.2.2 Hyperparameters	15
4.2.3 Quy trình tiền xử lý dữ liệu	15
4.3 Responsible AI	16
4.3.1 Tính công bằng (Fairness)	16
4.3.2 Khả năng giải thích (Explainability)	16
4.3.3 Quyền riêng tư và đạo đức	16

5	Quy trình Triển khai và Thực nghiệm	17
5.1	Môi trường triển khai	17
5.2	Các bước triển khai	17
5.2.1	Bước 1: Khởi động hạ tầng	17
5.2.2	Bước 2: Huấn luyện mô hình ban đầu	17
5.2.3	Bước 3: Demo drift và tái huấn luyện tự động	18
5.2.4	Bước 4: Giám sát và quan sát kết quả	18
5.3	Kiểm thử (Testing)	19
5.3.1	Cấu trúc bộ test	19
5.3.2	Chạy kiểm thử	19
5.4	Xử lý các thách thức kỹ thuật	19
6	Kết quả Đạt được	21
6.1	Kết quả huấn luyện mô hình	21
6.2	Kết quả phát hiện drift và tái huấn luyện	21
6.3	Kết quả kiểm thử tự động	22
6.4	Kết quả Airflow DAG	22
6.5	Kết quả Responsible AI	23
6.5.1	Fairness giữa hai loại rượu	23
6.5.2	Explainability	23
6.6	Kết quả giám sát hệ thống	24
6.6.1	Prometheus Metrics	24
6.6.2	Grafana Dashboard	24
6.6.3	Alerting	24
6.7	Kết quả CI/CD	25
6.8	So sánh với yêu cầu đề tài	25
7	Kết luận và Hướng Phát triển	26
7.1	Kết luận	26
7.2	Hướng phát triển	27
	Tài liệu tham khảo	28
A	Hướng dẫn cài đặt và chạy	29
A.1	Yêu cầu hệ thống	29
A.2	Các bước cài đặt từ đầu	29
B	Cấu hình Docker Compose	30
C	Cấu trúc Airflow DAG	32

Danh sách hình vẽ

2.1	Vòng đời của mô hình học máy trong MLOps	9
3.1	Danh sách các dịch vụ trong hệ thống	11
3.2	Airflow DAG – 6 tasks ML pipeline	13

Danh sách bảng

4.1	Thông tin tổng quan về dataset Wine Quality	14
4.2	Mô tả các features trong dataset Wine Quality	14
4.3	Lưới tìm kiếm hyperparameters của RandomForestClassifier	15
5.1	Thông tin môi trường triển khai	17
5.2	Danh sách URL truy cập các thành phần hệ thống	18
5.3	Danh sách test files và số lượng test cases	19
6.1	Kết quả chạy script <code>simulate_drift_and_retrain.py</code>	21
6.2	Kết quả thực thi Airflow DAG ở lần tái huấn luyện	22
6.3	Chỉ số theo từng loại rượu trên tập kiểm tra (272 đỏ, 793 trắng)	23
6.4	Khoảng cách giữa hai loại rượu (0 là hoàn toàn đồng đều)	23
6.5	Mức độ quan trọng của features với Version 2 (các feature còn lại có giá trị nhỏ hơn)	23
6.6	Một số alert rules	25
6.7	So sánh kết quả với mục tiêu đặt ra	25

Chương 1

Giới thiệu

1.1 Bối cảnh và động lực

Trong kỷ nguyên số hóa, các mô hình học máy (Machine Learning – ML) đang được triển khai ngày càng rộng rãi trong các hệ thống sản xuất thực tế. Tuy nhiên, việc đưa một mô hình từ giai đoạn nghiên cứu vào môi trường production và duy trì hiệu năng theo thời gian là một thách thức lớn. Đây chính là vấn đề mà MLOps (Machine Learning Operations) được sinh ra để giải quyết.

MLOps kết hợp các thực hành tốt nhất từ DevOps, DataOps và kỹ thuật học máy nhằm tạo ra một quy trình tự động, có thể tái lập và liên tục cải thiện. Theo báo cáo của Gartner (2023), hơn 80% các dự án ML không bao giờ đến được giai đoạn production do thiếu hạ tầng vận hành phù hợp. MLOps chính là cầu nối giúp thu hẹp khoảng cách này.

1.2 Mục tiêu đề tài

Đề tài đặt ra các mục tiêu cụ thể sau:

1. **Xây dựng pipeline ML hoàn chỉnh** từ ingestion đến serving với kiến trúc microservices dựa trên Docker.
2. **Triển khai MLflow** để theo dõi thí nghiệm, versioning mô hình và quản lý lifecycle.
3. **Phát triển REST API** với FastAPI để phục vụ dự đoán theo thời gian thực.
4. **Thiết lập hệ thống monitoring** với Prometheus + Grafana để theo dõi metrics về hiệu năng và sức khỏe hệ thống.
5. **Phát hiện data drift tự động** bằng Evidently AI và kích hoạt tái huấn luyện thông qua Airflow.
6. **Đảm bảo chất lượng code** với bộ test cases đạt coverage $\geq 80\%$.
7. **Tích hợp CI/CD** với GitHub Actions để tự động hóa kiểm thử và triển khai.
8. **Responsible AI**: phân tích tính công bằng (fairness) và khả năng giải thích (explainability) của mô hình.

1.3 Phạm vi thực hiện

Báo cáo tập trung vào các nội dung sau:

- **Bối cảnh giả định:** Một nhà sản xuất rượu cần sàng lọc nhanh các mẻ rượu trước khi đưa cho hội đồng nếm thử, vốn chậm và tốn kém. Mô hình đưa ra tín hiệu sàng lọc sơ bộ từ các chỉ số phòng thí nghiệm; quyết định cuối cùng vẫn thuộc về hội đồng.
- **Bài toán:** Phân loại nhị phân: rượu đạt chất lượng (điểm ≥ 6) hay không đạt, dựa trên 11 thuộc tính hóa lý.
- **Dataset:** UCI Wine Quality (Cortez và cộng sự, 2009) – 1.599 mẫu rượu đỏ và 4.898 mẫu rượu trắng, 11 features.
- **Mô hình:** Random Forest Classifier (sklearn).
- **Môi trường:** Docker Desktop trên Windows 11; Python 3.10 trong các container.
- **Thời gian thực hiện:** Tháng 9–10/2026.

1.4 Cấu trúc báo cáo

Báo cáo được tổ chức thành 7 chương:

- **Chương 1:** Giới thiệu tổng quan.
- **Chương 2:** Cơ sở lý thuyết về MLOps.
- **Chương 3:** Kiến trúc và thiết kế hệ thống.
- **Chương 4:** Mô tả dữ liệu, mô hình ML và Responsible AI.
- **Chương 5:** Quy trình triển khai và thực nghiệm.
- **Chương 6:** Kết quả đạt được.
- **Chương 7:** Kết luận và hướng phát triển.

Chương 2

Cơ sở lý thuyết

2.1 MLOps là gì?

MLOps (Machine Learning Operations) là tập hợp các thực hành, công cụ và quy trình nhằm chuẩn hóa và tự động hóa vòng đời của mô hình học máy trong môi trường sản xuất. MLOps kế thừa các nguyên tắc từ DevOps và áp dụng vào lĩnh vực học máy.

Data → **Model Training** → **Evaluation** → **Deployment** → **Monitoring**
→ **Re-training**

(Vòng lặp liên tục – Continuous ML)

Hình 2.1: Vòng đời của mô hình học máy trong MLOps

2.1.1 Các thành phần cốt lõi của MLOps

Experiment Tracking: Ghi lại các thí nghiệm, siêu tham số (hyperparameters), metrics và artifacts của mỗi lần huấn luyện.

Model Registry: Kho lưu trữ và quản lý phiên bản mô hình với các trạng thái (Staging, Production, Archived).

Pipeline Orchestration: Tự động hóa quy trình từ ingestion đến deployment bằng các công cụ như Airflow, Prefect, Kubeflow.

Model Serving: Phục vụ dự đoán qua REST API, gRPC hoặc batch inference.

Monitoring: Theo dõi hiệu năng mô hình, data drift và infrastructure health.

Continuous Training: Tự động tái huấn luyện khi phát hiện model degradation hoặc data drift.

2.2 Data Drift và Model Drift

Data Drift (hay *Dataset Shift*) là hiện tượng phân phối của dữ liệu đầu vào thay đổi theo thời gian so với dữ liệu huấn luyện. Đây là một trong những nguyên nhân chính dẫn đến suy giảm hiệu năng mô hình trong production.

Có ba loại drift phổ biến:

- **Covariate Shift:** Phân phối đầu vào $P(X)$ thay đổi nhưng $P(Y|X)$ giữ nguyên.
- **Label Shift:** Phân phối nhãn $P(Y)$ thay đổi.

- **Concept Drift:** Mỗi quan hệ $P(Y|X)$ thay đổi theo thời gian.

Trong đề tài này, chúng tôi tạo ra **Covariate Shift** thật thay vì thêm nhiễu nhân tạo: mô hình được huấn luyện trên rượu đỏ, sau đó dữ liệu production chuyển sang rượu trắng. Hai loại rượu có đặc tính hóa học khác nhau rõ rệt (rượu trắng có lượng đường dư và sulfur dioxide cao hơn nhiều), nên phân phối đầu vào $P(X)$ thay đổi trên hầu hết các đặc trưng.

2.3 Các công cụ sử dụng

2.3.1 MLflow

MLflow là một nền tảng mã nguồn mở do Databricks phát triển, cung cấp 4 thành phần chính: *Tracking*, *Projects*, *Models* và *Registry*. Trong đề tài, chúng tôi sử dụng *Tracking* và *Registry*.

2.3.2 Apache Airflow

Apache Airflow là công cụ workflow orchestration mã nguồn mở, cho phép định nghĩa, lên lịch và giám sát các pipeline dưới dạng DAG (Directed Acyclic Graph). Mỗi node trong DAG là một Task.

2.3.3 FastAPI

FastAPI là một web framework Python hiện đại, hiệu năng cao, dựa trên ASGI. FastAPI tự động sinh tài liệu Swagger UI và hỗ trợ kiểm tra kiểu dữ liệu với Pydantic.

2.3.4 Evidently AI

Evidently AI là thư viện Python chuyên biệt để đánh giá, kiểm tra và giám sát mô hình ML. Nó cung cấp các báo cáo về chất lượng dữ liệu, data drift và model performance.

2.3.5 Prometheus & Grafana

Prometheus là hệ thống time-series database dùng để thu thập metrics qua HTTP scraping. Grafana là công cụ trực quan hóa metrics, kết nối với Prometheus để xây dựng dashboards.

2.3.6 Docker & Docker Compose

Docker cho phép đóng gói ứng dụng cùng dependencies vào container, đảm bảo tính nhất quán giữa các môi trường. Docker Compose quản lý multi-container applications.

Chương 3

Kiến trúc và Thiết kế Hệ thống

3.1 Tổng quan kiến trúc

Hệ thống được thiết kế theo kiến trúc microservices, với 8 dịch vụ độc lập giao tiếp qua Docker network monitoring. Tất cả dịch vụ được định nghĩa trong file `docker-compose.yml`. Artifacts của mô hình được lưu trên một Docker volume của MLflow server và được phục vụ qua HTTP, nên các dịch vụ khác không cần mount chung thư mục nào.

Service	Port	Vai trò
postgres	5432	Database (Airflow metadata, MLflow)
mlflow	5000	Experiment tracking & Model registry
api (FastAPI)	8000	REST API serving dự đoán
evidently	8001	Data drift detection service
prometheus	9090	Metrics collection
grafana	3000	Metrics visualization (dashboard)
airflow-webserver	8080	Pipeline orchestration UI
airflow-scheduler	--	DAG scheduler (background)

Hình 3.1: Danh sách các dịch vụ trong hệ thống

3.2 Luồng dữ liệu tổng thể

Luồng hoạt động chính của hệ thống được mô tả như sau:

- Training phase:** Script `training.py` kích hoạt Airflow DAG `ml_training_pipeline` với dữ liệu rượu đỏ. DAG làm sạch dữ liệu, tinh chỉnh và huấn luyện mô hình Random Forest, log lên MLflow và đăng ký mô hình vào Production.
- Serving phase:** FastAPI load mô hình Production từ MLflow Registry và cung cấp endpoint `POST /predict` cho client.
- Monitoring phase:** Mỗi request được ghi lại metrics (latency, error rate, prediction distribution), Prometheus scrape mỗi 5 giây. Đồng thời API gửi nền mỗi inference (features và dự đoán) sang Evidently.
- Drift Detection:** Evidently so sánh các inference gần nhất với dữ liệu huấn luyện (reference) bằng kiểm định Kolmogorov–Smirnov trên từng đặc trưng. Dataset bị coi là drift khi tỷ lệ đặc trưng $\text{drift} \geq 30\%$.

5. **Auto-Retraining:** Khi phát hiện drift, chính dịch vụ Evidently gọi Airflow REST API để kích hoạt DAG, lần này trên toàn bộ dữ liệu (rượu đỏ và rượu trắng).
6. **New Model:** DAG so sánh mô hình mới với mô hình Production trên cùng một tập kiểm tra. Nếu mô hình mới không kém hơn, nó được đưa lên Production, API tự nạp lại mô hình và Evidently nhận dữ liệu huấn luyện mới làm reference.

3.3 Cấu trúc thư mục dự án

Listing 3.1: Cấu trúc thư mục dự án

```

1 DDM501_FinalProject/
2 |-- airflow/
3 |   |-- Dockerfile           # Airflow image + ML dependencies
4 |   |-- dags/
5 |       |-- ml_pipeline_dag.py # Airflow DAG 6 tasks
6 |-- api/
7 |   |-- Dockerfile           # Multi-stage, non-root
8 |   |-- main.py               # FastAPI application
9 |   |-- requirements.txt
10 |-- config/
11 |   |-- prometheus.yml        # Scrape config
12 |   |-- prometheus/           # Alert rules (24 rules)
13 |   |-- grafana/              # Dashboards + provisioning
14 |-- data/                     # winequality-red.csv,
    winequality-white.csv
15 |-- evidently/
16 |   |-- Dockerfile
17 |   |-- main.py               # Drift service + retrain trigger
18 |-- mlflow/
19 |   |-- Dockerfile           # MLflow server
20 |-- scripts/
21 |   |-- training.py           # Trigger initial training
22 |   |-- simulate_drift_and_retrain.py
23 |-- src/                       # Core ML modules
24 |   |-- data_pipeline.py
25 |   |-- feature_engineering.py
26 |   |-- train.py
27 |   |-- evaluate.py
28 |   |-- fairness.py
29 |   |-- explainability.py
30 |-- tests/                     # Unit, integration, data, model
    tests
31 |-- .github/workflows/
32 |   |-- ci-cd.yml             # GitHub Actions CI/CD
33 |-- ARCHITECTURE.md, RESPONSIBLE_AI.md, CONTRIBUTING.md
34 |-- docker-compose.yml

```

3.4 Thiết kế Airflow DAG

DAG `ml_training_pipeline` gồm 6 tasks thực thi tuần tự. Toàn bộ logic nằm trong các hàm của thư mục `src/`; DAG chỉ điều phối và chuyển dữ liệu giữa các task qua file.

```
data_ingestion → data_cleaning → feature_engineering → model_training  
→ model_evaluation_and_registration → deploy_model
```

Hình 3.2: Airflow DAG – 6 tasks ML pipeline

data_ingestion: Đọc file CSV theo loại rượu. Mặc định đọc cả hai loại; khi được kích hoạt với tham số `colors=["red"]` thì chỉ đọc rượu đỏ (dùng cho lần huấn luyện đầu).

data_cleaning: Kiểm tra schema (thiếu cột thì dừng), ép kiểu, điền giá trị thiếu, loại dòng trùng lặp, tạo nhãn nhị phân.

feature_engineering: Chia train/test 80/20 có stratify theo từng loại rượu, fit `StandardScaler` trên tập train và lưu thành artifact.

model_training: Grid search với 5-fold cross-validation, ghép scaler và mô hình tốt nhất thành một Pipeline, log tham số, metrics, báo cáo explainability và chỉ số fairness lên MLflow.

model_evaluation_and_registration: Chấm mô hình mới và mô hình Production hiện tại trên cùng tập test; đăng ký và đưa lên Production nếu accuracy không thấp hơn.

deploy_model: Chỉ chạy khi mô hình được promote: gọi API nạp lại mô hình và gửi tập train mới cho Evidently làm reference. Nếu không, task được bỏ qua (skipped).

3.5 Kiến trúc CI/CD

GitHub Actions workflow chạy trên self-hosted runner với 4 jobs nối tiếp nhau:

1. **lint:** Kiểm tra code style với flake8 và ruff.
2. **test:** Chạy pytest, dừng nếu coverage dưới 80%.
3. **build:** Build toàn bộ Docker images.
4. **deploy:** Chỉ trên nhánh main: khởi động lại các dịch vụ bằng Docker Compose và chờ API trả lời `/health`.

Chương 4

Mô tả Dữ liệu, Mô hình và Responsible AI

4.1 Dataset: Wine Quality

Bộ dữ liệu được sử dụng là *UCI Wine Quality* (Cortez và cộng sự, 2009), gồm kết quả phân tích hóa lý của các mẫu rượu *vinho verde* vùng Bắc Bồ Đào Nha, kèm điểm chất lượng do chuyên gia nếm thử chấm (thang 0–10). Dữ liệu gồm hai file riêng cho rượu đỏ và rượu trắng, được lưu trong thư mục `data/` của repository để pipeline không phụ thuộc mạng.

Bảng 4.1: Thông tin tổng quan về dataset Wine Quality

Thuộc tính	Rượu đỏ	Rượu trắng	Tổng
Số mẫu gốc	1.599	4.898	6.497
Số mẫu sau khi loại trùng lặp	1.359	3.961	5.320
Tỷ lệ mẫu đạt (điểm ≥ 6)	52,9%	66,0%	62,6%
Tập huấn luyện (80%)	1.087	3.168	4.255
Tập kiểm tra (20%)	272	793	1.065

Dữ liệu có 11 features kiểu số thực, không có giá trị thiếu. Điểm chất lượng thực tế nằm trong khoảng 3–9 và tập trung ở điểm 5 và 6, nên bài toán được đặt thành **phân loại nhị phân**: nhãn 1 (đạt) nếu điểm ≥ 6 , ngược lại là 0. Cách đặt này cho hai lớp tương đối cân bằng và tránh các lớp chỉ có vài mẫu (điểm 3 và 9).

4.1.1 Mô tả 11 features

Bảng 4.2: Mô tả các features trong dataset Wine Quality

STT	Feature	Mô tả
1	fixed_acidity	Độ axit cố định (g/dm^3)
2	volatile_acidity	Độ axit bay hơi (g/dm^3)
3	citric_acid	Axit citric (g/dm^3)
4	residual_sugar	Đường dư (g/dm^3)
5	chlorides	Clorua (g/dm^3)
6	free_sulfur_dioxide	Sulfur dioxide tự do (mg/dm^3)
7	total_sulfur_dioxide	Tổng sulfur dioxide (mg/dm^3)
8	density	Khối lượng riêng (g/cm^3)
9	pH	Độ pH
10	sulphates	Sunfat (g/dm^3)
11	alcohol	Nồng độ cồn (% thể tích)

4.2 Mô hình học máy

4.2.1 Lựa chọn mô hình

Chúng tôi sử dụng **Random Forest Classifier** từ thư viện scikit-learn vì các lý do:

- Xử lý tốt dữ liệu dạng bảng kích thước nhỏ mà không cần feature selection thủ công.
- Ít bị overfitting nhờ cơ chế ensemble averaging.
- Cung cấp feature importance và hỗ trợ SHAP TreeExplainer giúp giải thích mô hình.
- Không cần GPU, thời gian huấn luyện ngắn, phù hợp với việc tái huấn luyện tự động.

4.2.2 Hyperparameters

Hyperparameters được chọn bằng grid search với cross-validation trên tập train. Mỗi tổ hợp trong lưới được ghi thành một run con trên MLflow.

Bảng 4.3: Lưới tìm kiếm hyperparameters của RandomForestClassifier

Tham số	Giá trị thử	Giải thích
n_estimators	100, 300	Số cây quyết định
max_depth	5, 10, None	Độ sâu tối đa của cây
min_samples_leaf	1, 3	Số mẫu tối thiểu ở mỗi lá
random_state	42	Seed cố định để tái lập
Số tổ hợp	12	$2 \times 3 \times 2$
cv	5 folds	Stratified, có xáo trộn
test_size	0.2 (80/20)	Tỷ lệ train/test split

4.2.3 Quy trình tiền xử lý dữ liệu

1. **Load data:** Đọc file CSV theo loại rượu, thêm cột color.
2. **Làm sạch:** Kiểm tra đủ cột, ép kiểu float, điền giá trị thiếu bằng trung vị, loại 1.177 dòng trùng lặp (nếu để lại, cùng một mẫu có thể nằm ở cả tập train và tập test).
3. **Tạo nhãn:** target = 1 nếu quality ≥ 6 .
4. **Train-test split:** 80% train / 20% test, stratified theo nhãn và thực hiện riêng cho từng loại rượu. Nhờ vậy tập test rượu đỏ không thay đổi khi thêm rượu trắng, và mô hình cũ không bao giờ được huấn luyện trên các mẫu dùng để so sánh nó với mô hình mới.
5. **Feature scaling:** StandardScaler fit trên tập train. Scaler và mô hình được ghép thành một Pipeline duy nhất, nên mô hình được phục vụ nhận trực tiếp dữ liệu thô.

4.3 Responsible AI

4.3.1 Tính công bằng (Fairness)

Dữ liệu không chứa thông tin cá nhân nên không có thuộc tính nhạy cảm như giới tính hay tuổi. Nhóm cần quan tâm là **loại rượu**: một mô hình sai nhiều hơn một cách có hệ thống trên một loại rượu là không công bằng với nhà sản xuất loại rượu đó. Module `src/fairness.py` tính cho từng nhóm: accuracy, tỷ lệ được dự đoán “đạt” (selection rate), true positive rate (TPR) và false positive rate (FPR), cùng khoảng cách lớn nhất giữa các nhóm. Các chỉ số này được log lên MLflow ở mỗi lần huấn luyện. Kết quả được trình bày ở mục 6.5.

4.3.2 Khả năng giải thích (Explainability)

Module `src/explainability.py` dùng ba phương pháp bổ sung cho nhau:

- **Feature importance theo impurity**: mức độ rừng cây dùng mỗi feature để tách nút (toàn cục, tính từ quá trình huấn luyện).
- **Permutation importance**: mức giảm accuracy trên dữ liệu chưa thấy khi xáo trộn một feature (toàn cục, không phụ thuộc loại mô hình).
- **SHAP (TreeExplainer)**: đóng góp của từng feature vào từng dự đoán cụ thể (cục bộ), tổng hợp thành biểu đồ summary.

Báo cáo HTML cùng các biểu đồ được lưu thành artifact của mỗi run trên MLflow.

4.3.3 Quyền riêng tư và đạo đức

- **Quyền riêng tư**: Dữ liệu chỉ gồm chỉ số hóa lý của rượu, không có thương hiệu, nhà sản xuất hay giá bán. Hệ thống chỉ lưu tạm 10.000 inference gần nhất trong bộ nhớ của dịch vụ Evidently. API chưa có xác thực, chỉ phù hợp cho môi trường demo.
- **Vai trò của mô hình**: “Chất lượng” là đánh giá cảm quan của con người và khoảng 24% dự đoán là sai, nên mô hình chỉ đóng vai trò sàng lọc sơ bộ, không thay thế hội đồng nếm thử.
- **Nhãn mang tính chủ quan**: Điểm số đến từ một nhóm nhỏ chuyên gia của một vùng rượu; không nên dùng mô hình cho vùng rượu khác khi chưa có dữ liệu mới.
- **Rủi ro của tự động hóa**: Mô hình retrain chỉ được triển khai khi không kém mô hình Production trên cùng tập test; các phiên bản cũ được giữ lại trong registry để có thể quay lui.

Phân tích đầy đủ nằm trong file `RESPONSIBLE_AI.md` của repository.

Chương 5

Quy trình Triển khai và Thực nghiệm

5.1 Môi trường triển khai

Bảng 5.1: Thông tin môi trường triển khai

Thành phần	Chi tiết
Hệ điều hành	Windows 11
Docker Desktop	Docker Engine 27.x
Python	3.10 (trong container)
scikit-learn	1.5.2 (ghim giống nhau ở API và Airflow)
numpy	1.26.4 (ghim giống nhau ở API và Airflow)
MLflow	2.16.2 (build từ mlflow/Dockerfile)
Airflow	2.8.4 (build từ airflow/Dockerfile)
Evidently	0.4.26
FastAPI	0.104.1
Prometheus	v2.54.1
Grafana	11.2.0
PostgreSQL	15-alpine

5.2 Các bước triển khai

5.2.1 Bước 1: Khởi động hạ tầng

Listing 5.1: Khởi động toàn bộ 8 services bằng Docker Compose

```
1 # Tao file cau hinh tu file mau
2 cp .env.example .env
3
4 # Build va khoi dong tat ca services o che do daemon
5 docker compose up -d --build
6
7 # Kiem tra trang thai
8 docker compose ps
```

Sau khi chạy lệnh trên, 8 containers được khởi động và sẵn sàng sau khoảng 1–2 phút. Lúc này API báo unhealthy vì chưa có mô hình nào.

5.2.2 Bước 2: Huấn luyện mô hình ban đầu

Listing 5.2: Huấn luyện mô hình lần đầu

```
python scripts/training.py
```

Script `training.py` kích hoạt Airflow DAG với tham số chỉ dùng rượu đỏ và chờ DAG chạy xong. DAG thực hiện:

- Đọc và làm sạch dữ liệu rượu đỏ.
- Grid search và huấn luyện `RandomForestClassifier`.
- Log tham số, metrics (accuracy, precision, recall, F1, cross-validation), báo cáo explainability lên MLflow.
- Đăng ký mô hình `wine_quality_model` Version 1 vào stage **Production**.
- Gọi API nạp mô hình và gửi tập train cho Evidently làm reference.

5.2.3 Bước 3: Demo drift và tái huấn luyện tự động

Listing 5.3: Chạy demo data drift

```
python scripts/simulate_drift_and_retrain.py
```

Kịch bản mô phỏng:

1. Gửi **100 request rượu đỏ** đến API POST `/predict` và yêu cầu Evidently phân tích: không có drift.
2. Gửi **250 request rượu trắng**.
3. Evidently phát hiện drift ($\geq 30\%$ features bị drift) và tự kích hoạt Airflow DAG `ml_training_pipeline` qua REST API.
4. DAG chạy 6 tasks trên cả hai loại rượu; mô hình mới được so sánh với mô hình cũ và đăng ký phiên bản mới.
5. Script chờ DAG hoàn thành và xác nhận API đã chuyển sang phiên bản mô hình mới.

Script không tự gán kết quả: nếu Evidently không phát hiện drift hoặc không kích hoạt được DAG, script kết thúc với mã lỗi.

5.2.4 Bước 4: Giám sát và quan sát kết quả

Sau khi hệ thống chạy, người dùng có thể truy cập các giao diện:

Bảng 5.2: Danh sách URL truy cập các thành phần hệ thống

Dịch vụ	URL	Thông tin đăng nhập
MLflow Tracking	http://localhost:5000	Không cần
Airflow DAGs	http://localhost:8080	admin / admin
FastAPI Swagger	http://localhost:8000/docs	Không cần
Grafana Dashboard	http://localhost:3000	admin / admin
Prometheus Targets	http://localhost:9090/targets	Không cần
Evidently Service	http://localhost:8001	Không cần

5.3 Kiểm thử (Testing)

5.3.1 Cấu trúc bộ test

Bảng 5.3: Danh sách test files và số lượng test cases

File test	Loại test và nội dung	Số tests
test_data.py	Unit + data quality: schema, null, trùng lặp, nhãn, split, scaler	17
test_train.py	Unit: huấn luyện, tuning, log MLflow (mock)	15
test_evaluate.py	Unit: đánh giá, quy tắc promote, registry (mock)	15
test_model.py	Model validation: accuracy tối thiểu, tái lập, số lớp	7
test_api.py	Integration: ứng dụng FastAPI thật, mọi endpoint và nhánh lỗi	24
test_explainability.py	Unit: feature importance, permutation importance, SHAP	12
test_fairness.py	Unit: chỉ số fairness theo nhóm	6
Tổng cộng		96

5.3.2 Chạy kiểm thử

Listing 5.4: Chạy bộ test với coverage report

```

1  # Cài đặt dependencies
2  pip install -r requirements.txt
3
4  # Chạy toàn bộ test với coverage
5  pytest tests/ --cov=src --cov=api --cov-report=term-missing -v
6
7  # Kết quả: 96 tests PASSED -- coverage: 96%
```

5.4 Xử lý các thách thức kỹ thuật

Trong quá trình triển khai, nhóm đã gặp và giải quyết một số thách thức kỹ thuật đáng chú ý:

- Lưu trữ artifacts của MLflow:** Ban đầu hệ thống dùng MinIO làm artifact store, nhưng image minio/minio không còn pull được từ Docker Hub. Giải pháp tạm thời là mount thư mục C:/mlflow vào mọi container, cách này phụ thuộc đường dẫn của máy host và gây lỗi Permission denied ở Airflow. Giải pháp cuối cùng: MLflow server lưu artifacts trên một Docker volume và phục vụ qua HTTP (--serve-artifacts), các dịch vụ khác không cần mount gì.
- Lệch phiên bản NumPy giữa nơi huấn luyện và nơi phục vụ:** Mô hình pickle bởi NumPy 2.x không đọc được ở container dùng NumPy 1.x (lỗi No module named 'numpy._core'). Giải pháp: mọi lần huấn luyện đều chạy trong container Airflow, và numpy, scikit-learn được ghim cùng phiên bản ở Airflow và API.

3. **Lệch train/serve:** Mô hình được huấn luyện trên dữ liệu đã chuẩn hóa nhưng API gửi dữ liệu thô. Giải pháp: ghép scaler và mô hình thành một Pipeline và đăng ký cả pipeline lên MLflow.
4. **Evidently báo drift sai trên dữ liệu bình thường:** Khi reference có hơn 1.000 dòng, Evidently mặc định dùng khoảng cách Wasserstein và báo drift 6–10/11 features ngay cả với 100 mẫu rượu đỏ bình thường. Giải pháp: chỉ định kiểm định Kolmogorov–Smirnov; khi đó dữ liệu bình thường chỉ còn 0–2 features bị báo drift, còn rượu trắng là 10–11.
5. **Cross-validation bị chia theo loại rượu:** Tập train được nối từ rượu đỏ rồi rượu trắng, nên các fold không xáo trộn gần như chỉ chứa một loại rượu (độ lệch chuẩn của CV accuracy là 0,086). Giải pháp: dùng StratifiedKFold có xáo trộn (độ lệch chuẩn còn 0,010).
6. **Ký tự xuống dòng trên Windows:** Script init-db.sh bị Git chuyển sang CRLF khiến container PostgreSQL không khởi động (bad interpreter). Giải pháp: thêm .gitattributes buộc các file .sh dùng LF. Tương tự, emoji trong lệnh print gây UnicodeEncodeError trên terminal Windows, được xử lý bằng `sys.stdout.reconfigure(encoding='utf-8')`.

Chương 6

Kết quả Đạt được

6.1 Kết quả huấn luyện mô hình

Kết quả

Version 1 – huấn luyện ban đầu trên rượu đỏ (1.087 mẫu train, 272 mẫu test)

Hyperparameters tốt nhất: `n_estimators=300`, `max_depth=10`, `min_samples_leaf=1`

Accuracy (test set rượu đỏ)	77,6%
Precision / Recall / F1 (weighted)	0,777 / 0,776 / 0,776
Cross-validation (5-fold)	76,0% $\pm$ 4,1%

Version 2 – tái huấn luyện trên rượu đỏ + trắng (4.255 mẫu train, 1.065 mẫu test)

Hyperparameters tốt nhất: `n_estimators=300`, `max_depth=None`, `min_samples_leaf=3`

Accuracy (test set đỏ + trắng)	76,1%
Precision / Recall / F1 (weighted)	0,759 / 0,761 / 0,760
Cross-validation (5-fold)	76,9% $\pm$ 1,0%
Version 1 trên cùng test set	70,7%

Ghi chú

Độ chính xác khoảng 76% là mức thực tế của bài toán này: nhãn là đánh giá cảm quan của con người và ranh giới giữa điểm 5 và điểm 6 không rõ ràng. Điều quan trọng với hệ thống là **mức chênh lệch**: Version 1 chỉ đạt 70,7% khi dữ liệu có thêm rượu trắng, còn Version 2 đạt 76,1% trên cùng tập kiểm tra. Đây là căn cứ để pipeline tự động đưa Version 2 lên Production.

6.2 Kết quả phát hiện drift và tái huấn luyện

Bảng 6.1: Kết quả chạy script `simulate_drift_and_retrain.py`

Bước	Dữ liệu gửi đến API	Kết quả
1	100 mẫu rượu đỏ (bình thường)	0/11 features drift – không drift
2–3	250 mẫu rượu trắng	10/11 features drift – có drift, kích hoạt DAG
4	–	DAG thành công, API chuyển từ Version 1 sang Version 2

Trên MLflow, run của lần tái huấn luyện được gắn tag cho biết nó do Evidently kích hoạt. Run này ghi cả accuracy của mô hình mới (0,761) và accuracy của mô hình Production cũ trên cùng tập kiểm tra (0,707), cho thấy rõ lý do mô hình được promote. Sau khi triển khai, reference của Evidently được cập nhật từ 1.087 dòng rượu đỏ thành 4.255 dòng của cả hai loại.

6.3 Kết quả kiểm thử tự động

Kết quả

Tổng số test cases: 96

Kết quả: 96 PASSED, 0 FAILED, 0 ERROR

Code Coverage: 96% trên src/ và api/ (yêu cầu $\geq 80\%$)

Thời gian chạy: ~100 giây

6.4 Kết quả Airflow DAG

Sau khi Evidently phát hiện data drift, Airflow DAG được kích hoạt và hoàn thành thành công:

Bảng 6.2: Kết quả thực thi Airflow DAG ở lần tái huấn luyện

Task	Trạng thái	Duration	Ghi chú
data_ingestion	SUCCESS	~0,4s	Đọc 6.497 mẫu
data_cleaning	SUCCESS	~0,4s	Còn 5.320 mẫu
feature_engineering	SUCCESS	~0,3s	Split, fit scaler
model_training	SUCCESS	~80s	Grid search 12 tổ hợp
model_evaluation_and_registration	SUCCESS	~2s	Promote Version 2
deploy_model	SUCCESS	~0,7s	Reload API, reference
Tổng	6/6 SUCCESS	~84s	

Lần huấn luyện ban đầu (chỉ rượu đỏ) mất khoảng 43 giây, trong đó model_training chiếm 39 giây.

6.5 Kết quả Responsible AI

6.5.1 Fairness giữa hai loại rượu

Bảng 6.3: Chỉ số theo từng loại rượu trên tập kiểm tra (272 đỏ, 793 trắng)

Mô hình	Loại rượu	Accuracy	Dự đoán đạt	TPR	FPR
Version 1 (chỉ đỏ)	Đỏ	0,776	0,511	0,771	0,219
	Trắng	0,683	0,562	0,686	0,322
Version 2 (đỏ + trắng)	Đỏ	0,783	0,504	0,771	0,203
	Trắng	0,753	0,687	0,834	0,404

Bảng 6.4: Khoảng cách giữa hai loại rượu (0 là hoàn toàn đồng đều)

Tiêu chí	Version 1	Version 2
Accuracy	0,092	0,030
Equal opportunity (TPR)	0,084	0,063
Demographic parity (tỷ lệ dự đoán đạt)	0,051	0,184
False positive rate	0,103	0,201

Version 1 có thiên lệch do thiếu đại diện (representation bias): trên rượu trắng nó chỉ đạt 68,3%, gần bằng việc luôn trả lời “đạt” (66,0%). Tái huấn luyện là biện pháp giảm thiên lệch: khoảng cách accuracy giảm từ 0,092 xuống 0,030 mà không làm giảm kết quả trên rượu đỏ.

Tuy nhiên Version 2 chưa công bằng theo mọi tiêu chí. Khoảng cách FPR tăng gấp đôi: 40,4% rượu trắng không đạt bị dự đoán là đạt, so với 20,3% ở rượu đỏ. Với một công cụ sàng lọc, đây là loại lỗi tốn kém. Khoảng cách demographic parity tăng chủ yếu vì hai loại rượu khác nhau thật (66,0% rượu trắng đạt so với 52,9% rượu đỏ). Các hướng khắc phục chưa thực hiện gồm: ngưỡng quyết định riêng cho từng loại rượu, thêm loại rượu làm feature, và bổ sung điều kiện về khoảng cách fairness vào quy tắc promote.

6.5.2 Explainability

Bảng 6.5: Mức độ quan trọng của features với Version 2 (các feature còn lại có giá trị nhỏ hơn)

Feature	Permutation importance	Impurity importance
alcohol	0,081	0,207
volatile_acidity	0,053	0,114
free_sulfur_dioxide	0,020	0,082
sulphates	0,015	0,076
total_sulfur_dioxide	0,014	0,076
density	0,003	0,107

Cả hai phương pháp đều cho thấy `alcohol` và `volatile_acidity` là hai yếu tố quyết định, phù hợp với hiểu biết về rượu vang (độ axit bay hơi cao gây vị giấm). Hai phương pháp bất đồng về `density`: rừng cây dùng nó nhiều để tách nút (0,107) nhưng xáo trộn nó gần như không làm giảm `accuracy` (0,003), vì `density` gần như được xác định bởi nồng độ cồn và lượng đường. Đây là lý do không nên chỉ dùng `impurity importance`.

6.6 Kết quả giám sát hệ thống

6.6.1 Prometheus Metrics

Prometheus thu thập các metrics sau từ FastAPI (chu kỳ 5 giây) và từ Evidently:

- `api_requests_total`: Tổng số requests theo endpoint và status code.
- `api_request_latency_seconds`: Phân phối latency theo histogram.
- `model_predictions_total`: Số dự đoán theo phiên bản mô hình.
- `model_prediction_latency_seconds`: Thời gian inference.
- `model_prediction_value`: Phân phối giá trị dự đoán (0 hoặc 1).
- `model_prediction_errors_total`: Số lỗi dự đoán theo loại lỗi.
- `evidently_data_drift_detected`: Dataset có bị drift hay không.
- `evidently_drifted_features_count`: Số features bị drift.
- `evidently_feature_drift`: Trạng thái drift của từng feature.

Trong lần chạy demo (350 requests), latency p95 của `/predict` vào khoảng 50 ms. Đây là số đo trên máy cục bộ, không phải kết quả load test.

6.6.2 Grafana Dashboard

Grafana được cấu hình sẵn (provisioning) hai dashboard:

1. **ML Model Monitoring** (9 panels): số request/giây, latency P50/P95/P99, tỷ lệ lỗi, request theo status code, số dự đoán/giây, latency của mô hình và phân phối dự đoán.
2. **Evidently – Data Drift Monitoring** (11 panels): trạng thái drift, số features bị drift, drift score theo thời gian, ma trận drift theo feature, thời gian và tần suất phân tích, tỷ lệ giá trị thiếu.

6.6.3 Alerting

Prometheus nạp 24 alert rules từ thư mục `config/prometheus/`: 6 rules cho API và mô hình, 18 rules cho drift và chất lượng dữ liệu. Một số rule tiêu biểu:

Bảng 6.6: Một số alert rules

Alert	Điều kiện	Mức độ
ModelAPIDown	Prometheus không scrape được API trong 1 phút	critical
ModelNotLoaded	Có dự đoán bị từ chối vì API chưa nạp mô hình	critical
HighErrorRate	Hơn 5% requests trả về lỗi 5xx trong 5 phút	critical
HighPredictLatency	Latency p95 của /predict trên 500 ms	warning
DataDriftDetected	Evidently báo dataset drift	warning

6.7 Kết quả CI/CD

GitHub Actions workflow được cấu hình tại `.github/workflows/ci-cd.yml`:

- **Trigger:** Push lên branch main hoặc develop, hoặc mở Pull Request vào main.
- **Jobs:** Lint → Test (với coverage check) → Build → Deploy (chỉ trên main).
- **Runner:** self-hosted.

Các bước lint và test của workflow đã được chạy cục bộ với cùng lệnh và đều đạt (flake8 và ruff không có lỗi, 96 tests pass).

6.8 So sánh với yêu cầu đề tài

Bảng 6.7: So sánh kết quả với mục tiêu đặt ra

Tiêu chí	Mục tiêu	Kết quả
Model accuracy (test set)	$\geq 75\%$	76,1%
Cross-validation accuracy	$\geq 75\%$	76,9% $\pm$ 1,0%
Chênh lệch accuracy giữa hai loại rượu	$\leq 0,05$	0,030
Latency p95 của /predict	< 500 ms	~50 ms
Code coverage	$\geq 80\%$	96%
Loại test	4 loại	Unit, integration, data, model
Drift detection	Có	Có (Evidently AI)
Auto-retraining	Có	Có (Airflow, 6 tasks)
REST API serving	Có	Có (FastAPI)
Monitoring và alerting	Có	2 dashboards, 24 rules
CI/CD pipeline	Có	Có (GitHub Actions)
Responsible AI	Có	Fairness, explainability

Chương 7

Kết luận và Hướng Phát triển

7.1 Kết luận

Đề tài đã xây dựng một hệ thống MLOps hoàn chỉnh, đáp ứng các yêu cầu của môn học. Cụ thể:

1. **Pipeline ML tự động:** Từ huấn luyện, đánh giá, đăng ký đến triển khai mô hình đều do Airflow DAG thực hiện; không có bước thủ công nào giữa lúc phát hiện drift và lúc mô hình mới được phục vụ.
2. **Kiến trúc microservices:** 8 dịch vụ Docker chạy độc lập, khởi động bằng một lệnh `docker compose up`.
3. **Vòng lặp khép kín:** Hệ thống tự phát hiện data drift, tái huấn luyện và chỉ triển khai mô hình mới khi nó không kém mô hình cũ trên cùng tập kiểm tra. Trong thực nghiệm, mô hình mới cải thiện accuracy từ 70,7% lên 76,1%.
4. **Chất lượng code:** 96 test cases với 96% coverage, vượt qua yêu cầu tối thiểu 80%.
5. **Khả năng quan sát (Observability):** Prometheus, Grafana và 24 alert rules cung cấp visibility về hiệu năng hệ thống, mô hình và drift.
6. **Responsible AI:** Chỉ số fairness theo loại rượu và báo cáo explainability được tạo tự động ở mỗi lần huấn luyện.

Hệ thống còn các hạn chế sau:

- Dữ liệu production không có nhãn, nên việc tái huấn luyện dùng dữ liệu đã có nhãn sẵn (file rượu trắng đóng vai trò “dữ liệu mới đã được gán nhãn”). Trong triển khai thật, điểm của hội đồng ném thử sẽ đến sau và đi vào cùng pipeline.
- Phân tích drift chỉ chạy khi endpoint `/analyze` được gọi, chưa chạy theo lịch.
- Quy tắc promote chỉ xét accuracy tổng thể, chưa xét các khoảng cách fairness; tỷ lệ dương tính giả trên rượu trắng còn cao (40,4%).
- Các kết quả đến từ một lần chia train/test, chưa kiểm định ý nghĩa thống kê.

Qua quá trình thực hiện, nhóm đã tích lũy kinh nghiệm quý báu về:

- Cách thiết kế và debug hệ thống multi-container phức tạp.
- Xử lý các vấn đề tương thích phiên bản trong stack Python/numpy/sklearn.
- Vận hành Apache Airflow cho ML pipeline orchestration.
- Áp dụng các kỹ thuật phát hiện data drift trong thực tế.

7.2 Hướng phát triển

Hệ thống hiện tại có thể được phát triển theo các hướng sau:

1. **Triển khai lên Kubernetes (K8s):** Sử dụng Helm charts để deploy lên cloud (GCP, AWS, Azure) với khả năng auto-scaling.
2. **Tích hợp Feature Store:** Sử dụng Feast hoặc Hopsworks để quản lý features tập trung, chia sẻ giữa nhiều mô hình.
3. **A/B Testing:** Triển khai nhiều phiên bản mô hình song song với traffic splitting để so sánh hiệu năng trong production.
4. **Hyperparameter Optimization tự động:** Tích hợp Optuna hoặc Ray Tune vào DAG để tự động tìm kiếm hyperparameters tối ưu khi retrain.
5. **Data versioning:** Sử dụng DVC (Data Version Control) để versioning dữ liệu song song với model versioning.
6. **Mô hình phức tạp hơn:** Áp dụng với deep learning (TensorFlow/PyTorch) và các bài toán phức tạp hơn như NLP, time-series forecasting.
7. **Security:** Thêm authentication cho các API endpoints, mã hóa secrets với HashiCorp Vault, và network policies cho Docker.

Tài liệu tham khảo

- [1] Sculley, D., et al. (2015). *Hidden Technical Debt in Machine Learning Systems*. NIPS 2015.
- [2] Zaharia, M., et al. (2018). *Accelerating the Machine Learning Lifecycle with MLflow*. VLDB 2018.
- [3] Apache Software Foundation. (2024). *Apache Airflow Documentation*. <https://airflow.apache.org/docs/>
- [4] Tikhonov, M., et al. (2022). *Evidently: An Open-Source Framework for Data and ML Monitoring*. <https://docs.evidentlyai.com/>
- [5] FastAPI Documentation. (2024). <https://fastapi.tiangolo.com/>
- [6] Prometheus Documentation. (2024). *Prometheus Monitoring System & Time Series Database*. <https://prometheus.io/docs/>
- [7] Grafana Labs. (2024). *Grafana Documentation*. <https://grafana.com/docs/>
- [8] Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). *Modeling wine preferences by data mining from physicochemical properties*. *Decision Support Systems*, 47(4), 547–553. Dữ liệu: <https://archive.ics.uci.edu/dataset/186/wine+quality>
- [9] Breck, E., et al. (2017). *The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction*. IEEE Big Data 2017.
- [10] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- [11] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
- [12] Alla, S., & Adari, S. K. (2021). *Beginning MLOps with MLflow*. Apress.

Chương A

Hướng dẫn cài đặt và chạy

A.1 Yêu cầu hệ thống

- Docker Desktop (đã bật và đang chạy)
- Python 3.10+
- Git

A.2 Các bước cài đặt từ đầu

Listing A.1: Cài đặt và chạy hệ thống từ đầu

```
1  # 1. Clone repository
2  git clone https://github.com/phamminhhoang1985/
   DDM501_FinalProject_25MS13285_PhamMinhHoang.git
3  cd DDM501_FinalProject_25MS13285_PhamMinhHoang
4
5  # 2. Tao file cau hinh (doi cong trong .env neu bi trung)
6  cp .env.example .env
7
8  # 3. Build va khoi dong services
9  docker compose up -d --build
10
11 # 4. Cai dat Python dependencies cho scripts
12 pip install requests colorama
13
14 # 5. Huan luyen mo hinh ban dau (tu cho Airflow chay xong)
15 python scripts/training.py
16
17 # 6. Chay demo drift & auto-retrain
18 python scripts/simulate_drift_and_retrain.py
19
20 # 7. Chay bo test
21 pip install -r requirements.txt
22 pytest tests/ --cov=src --cov=api -v
```

Chương B

Cấu hình Docker Compose

Listing B.1: Trích đoạn docker-compose.yml (services chính)

```
1  services:
2    postgres:
3      image: postgres:15-alpine
4      environment:
5        POSTGRES_USER: mlflow
6        POSTGRES_PASSWORD: mlflow
7        POSTGRES_DB: mlflow
8      ports: ["${POSTGRES_PORT:-5432}:5432"]
9
10   mlflow:
11     build: ./mlflow
12     ports: ["${MLFLOW_PORT:-5000}:5000"]
13     volumes:
14       - mlflow_data:/mlflow
15
16   api:
17     build: ./api
18     ports: ["${API_PORT:-8000}:8000"]
19     environment:
20       - MLFLOW_TRACKING_URI=http://mlflow:5000
21       - MODEL_NAME=wine_quality_model
22       - MODEL_STAGE=Production
23       - EVIDENTLY_URL=http://evidently:8001
24
25   evidently:
26     build: ./evidently
27     ports: ["${EVIDENTLY_PORT:-8001}:8001"]
28     environment:
29       - EVIDENTLY_DRIFT_THRESHOLD=0.3
30       - AIRFLOW_URL=http://airflow-webserver:8080
31
32   prometheus:
33     image: prom/prometheus:v2.54.1
34     volumes:
35       - ./config/prometheus.yml:/etc/prometheus/prometheus.yml:ro
36       - ./config/prometheus:/etc/prometheus/rules:ro
37
38   airflow-scheduler:
39     build: ./airflow
40     environment:
41       AIRFLOW__CORE__EXECUTOR: LocalExecutor
```

```
42     MLFLOW_TRACKING_URI: http://mlflow:5000
43     volumes:
44     - ./airflow/dags:/opt/airflow/dags
45     - ./src:/opt/airflow/src
46     - ./data:/opt/airflow/data
```

Chương C

Cấu trúc Airflow DAG

Listing C.1: ml_pipeline_dag.py – trích đoạn DAG chính

```
1 def model_evaluation_and_registration(**kwargs):
2     test_df = load_data_artifact(TEST_PATH)
3     X_test, y_test = test_df[FEATURE_NAMES], test_df[TARGET]
4
5     candidate = mlflow.sklearn.load_model(f"runs:{run_id}/model")
6     candidate_metrics = evaluate_model(candidate, X_test, y_test)
7
8     # Candidate and production are compared on the same test set
9     production = load_production_model()
10    promote = True
11    if production is not None:
12        production_metrics = evaluate_model(production, X_test,
13                                           y_test)
14        promote = should_promote(candidate_metrics,
15                                 production_metrics)
16
17    if promote:
18        register_and_promote(run_id)
19    return promote
20
21 with DAG(
22     dag_id='ml_training_pipeline',
23     default_args=default_args,
24     schedule_interval=None, # Chỉ chạy khi được trigger
25     start_date=datetime(2024, 1, 1),
26     catchup=False,
27     max_active_runs=1,
28 ) as dag:
29
30     t1 = PythonOperator(task_id='data_ingestion',
31                        python_callable=data_ingestion)
32     t2 = PythonOperator(task_id='data_cleaning',
33                        python_callable=data_cleaning)
34     t3 = PythonOperator(task_id='feature_engineering',
35                        python_callable=feature_engineering)
36     t4 = PythonOperator(task_id='model_training',
37                        python_callable=model_training)
38     t5 = PythonOperator(task_id='model_evaluation_and_registration',
```

```
38         python_callable=  
           model_evaluation_and_registration)  
39     t6 = PythonOperator(task_id='deploy_model',  
40                       python_callable=deploy_model)  
41  
42     t1 >> t2 >> t3 >> t4 >> t5 >> t6
```